

Propagação de Autorização em Hierarquia Delegada

Um modelo algébrico de controle distribuído sobre CASL,
com *enforcement* por *Row-Level Security* no *data layer*

Patria Technology — patria-erp

Maio de 2026

Resumo

Modelamos o controle de acesso do **patria-erp** como um *controle distribuído delegado*: um operador na raiz que enxerga tudo, cada organização (*tenant*) nascendo com controle total dos próprios recursos, e admins que criam perfis e delegam acesso aos seus associados, encadeando indefinidamente. A tese central é **desacoplar o *tenant* de qualquer tratamento especial e entregar todo o controle ao CASL**: o *tenant* passa a ser apenas *uma condição*. A chave é **propagar as condições (conditions) da raiz da hierarquia até o perfil-folha**. Mostramos que essa propagação é o *meet* (conjunção) ao longo do caminho na álgebra booleana dos conjuntos de linhas, que ela satisfaz uma invariante de não-escalada de privilégio, e que o *enforcement* se reduz a injetar a condição efetiva no **where** das consultas (*pushdown*), realizado por uma extensão do Prisma Client. O Apêndice A traz o plano de implementação.

Sumário

1	Introdução e motivação	1
2	Preliminares	2
3	Autorização num nó	2
4	A árvore de delegação e o operador de propagação	3
4.1	Modelo gráfico da propagação	3
5	Invariante de segurança e teoremas	3
6	O tenant como corolário (desacoplamento)	5
7	<i>Enforcement: pushdown no data layer</i>	5

1 Introdução e motivação

O sistema é multi-organização: cada cliente (um *tenant*) opera os próprios cadastros, vendas, alunos, mensalidades etc. A Patria, como operadora da plataforma, precisa *enxergar e operar* sobre os clientes de forma central; cada cliente precisa controlar o acesso dos seus funcionários; e um funcionário pode, por sua vez, ter um escopo ainda mais estreito. Isso é uma *hierarquia de delegação*.

A abordagem ingênua trata o *tenant* como um eixo especial — um “pisso” fixo aplicado a toda consulta (**where:** { **tenantId** }). Isso funciona, mas (i) espalha a responsabilidade de isolamento por centenas de *services*, onde é fácil esquecer; e (ii) não expressa a delegação encadeada. Adotamos a posição oposta: **o *tenant* não é especial**. Ele é apenas uma *condição* $c_t = (\text{tenantId} = t)$ como qualquer outra. Todo o controle — inclusive o isolamento de *tenant* — vive no CASL, e a hierarquia emerge da **propagação de condições**.

2 Preliminares

Definição 1 (Linhas e condições). Seja $\mathcal{R}$ o universo de todas as linhas (registros) de todas as tabelas. Cada $r \in \mathcal{R}$ é um registro com atributos ($r.\text{tenantId}$, $r.\text{id}$, $r.\text{branchId}$, ...). Uma *condição* c é um predicado $c : \mathcal{R} \rightarrow \{\perp, \top\}$, identificado ao seu *extent*

$$\llbracket c \rrbracket = \{ r \in \mathcal{R} : c(r) \} \subseteq \mathcal{R}.$$

Observação 2 (A álgebra booleana das condições). As condições, vistas pelos seus extents, formam a álgebra booleana $(2^{\mathcal{R}}, \cap, \cup, \setminus, \emptyset, \mathcal{R})$. A conjunção lógica $c \wedge c'$ é $\llbracket c \rrbracket \cap \llbracket c' \rrbracket$; a disjunção é a união; a negação é o complemento. Estas são exatamente as *MongoQuery* do CASL (objetos do tipo { **field:** **value**, **field2:** { **in:** [...] } }), fechadas sob AND/OR/NOT.

Fixamos um conjunto de *subjects* (recursos) $\mathcal{S}$ (**Customer**, **Product**, **Student**, ...) e de *ações* $\mathcal{A} = \{\text{create}, \text{read}, \text{update}, \text{delete}\}$, com **manage** = $\bigvee \mathcal{A}$ e o curinga **all** sobre $\mathcal{S}$.

Definição 3 (Regra). Uma *regra* CASL é uma tupla $\rho = (a, s, \phi, c, \iota)$ com ação a , subject s , conjunto de campos ϕ (opcional), condição c e *flag* de inversão $\iota \in \{0, 1\}$ ($\iota = 1$ é um **cannot**). Uma regra com $\iota = 0$ é um *grant*; com $\iota = 1$, uma *proibição*.

3 Autorização num nó

Num único nó (um perfil), para um par fixo (s, a) , as regras combinam na álgebra booleana: os *grants* **unem** (OR) e as *proibições* **subtraem** (AND NOT).

Definição 4 (Autorização local). Seja $\Phi_v(s, a)$ o conjunto de regras do nó v aplicáveis a (s, a) (casando subject e ação, com **manage/all** como curingas). A *autorização local* de v é

$$G_v(s, a) = \left(\bigcup_{\substack{\rho \in \Phi_v(s, a) \\ \iota_\rho = 0}} \llbracket c_\rho \rrbracket \right) \setminus \left(\bigcup_{\substack{\rho \in \Phi_v(s, a) \\ \iota_\rho = 1}} \llbracket c_\rho \rrbracket \right).$$

Observação 5 (Correção de um erro comum). Compor *grants* com interseção (AND) é um erro: dois grants “ler onde **org** = A ” e “ler onde **org** = B ” deveriam autorizar $A \cup B$, mas o AND produz $A \cap B = \emptyset$. A composição correta de *grants* é a **união**; apenas as *proibições* usam diferença.

4 A árvore de delegação e o operador de propagação

Definição 6 (Árvore de delegação). Seja T uma árvore (ou floresta) cuja raiz é o *operador* da plataforma. Os nós são *principals*/perfis, e as arestas são delegações:

$$\text{raiz} \rightarrow \text{dono-do-tenant} \rightarrow \dots \rightarrow \text{perfil-folha} \rightarrow \text{usuário}.$$

Cada nó v *introduz* uma condição local de escopo c_v (o seu refinamento sobre o que herdou).

Definição 7 (Condição efetiva por propagação). Para uma folha (perfil atribuído a um usuário) e um par (s, a) , a *condição efetiva* é a conjunção (o *meet*) ao longo do caminho da raiz à folha, restrita pela autorização local da folha:

$$c_{\text{eff}}(\text{folha}, s, a) = \left(\bigwedge_{v \in \text{path}(\text{raiz} \rightarrow \text{folha})} c_v \right) \wedge G_{\text{folha}}(s, a).$$

Equivalentemente, em extents, $\llbracket c_{\text{eff}} \rrbracket = \left(\bigcap_v \llbracket c_v \rrbracket \right) \cap G_{\text{folha}}(s, a)$.

Os papéis caem como casos particulares:

- **Raiz / operador:** $c_{\text{raiz}} = \top$, isto é $\llbracket c_{\text{raiz}} \rrbracket = \mathcal{R}$ — *vê tudo*.
- **Nó do tenant t :** introduz $c_t = (\text{tenantId} = t)$, confinando tudo abaixo ao tenant. *O tenant é apenas uma condição:* o desacoplamento.
- **Perfil de gerente:** pode introduzir $(\text{branchId} = b)$, $(\text{sellerId} = u)$, etc.
- **Usuário-folha:** recebe a conjunção do caminho com os grants do seu perfil.

4.1 Modelo gráfico da propagação

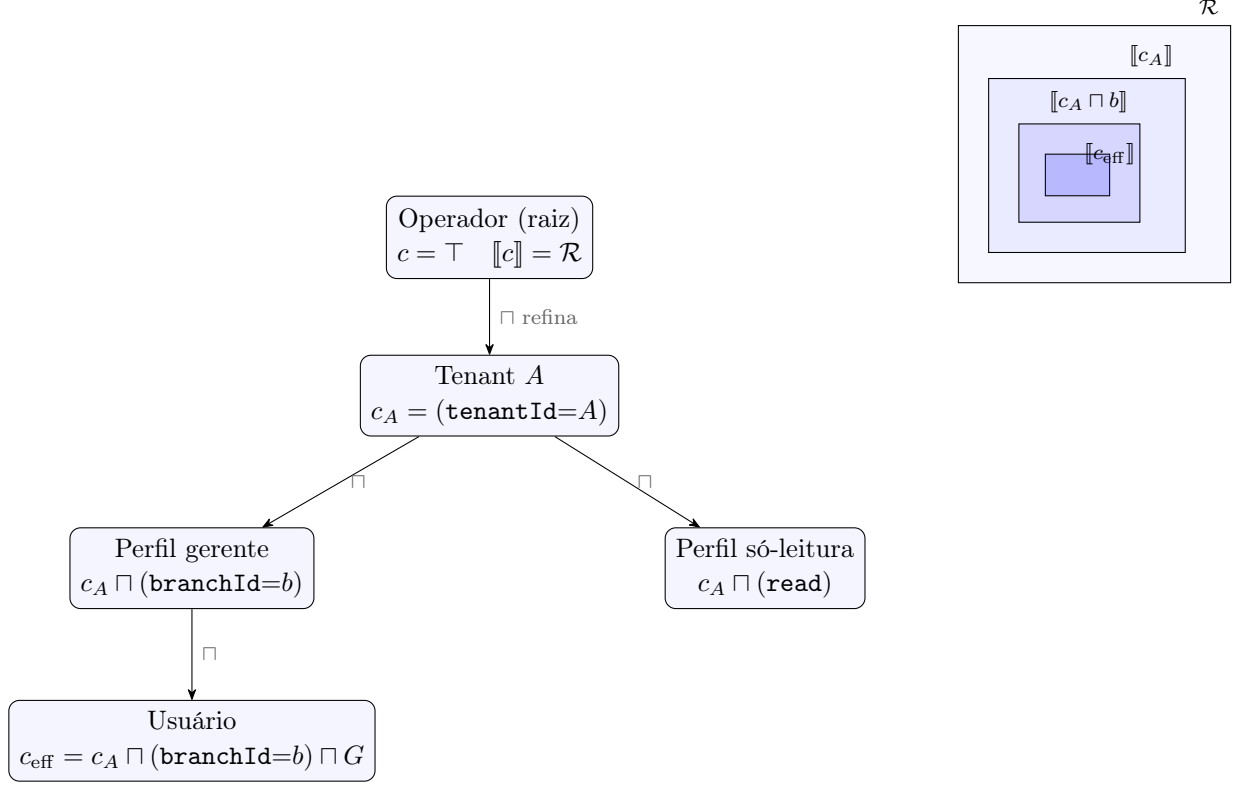


Figura 1: Propagação raiz→folha: cada aresta aplica o *meet* ($\sqcap$) com a condição introduzida no nó; os extents encolhem monotonicamente (caixas aninhadas à direita). A raiz ($\top$) vê $\mathcal{R}$; o nó do tenant confina a $\llbracket c_A \rrbracket$; perfis refinam até $\llbracket c_{\text{eff}} \rrbracket$.

5 Invariante de segurança e teoremas

Lema 8 (Monotonicidade da propagação). *Para qualquer nó w no caminho da raiz à folha, com pai v ,*

$$\llbracket c_{\text{eff}}(w) \rrbracket \subseteq \llbracket c_{\text{eff}}(v) \rrbracket.$$

Demonstração. Por Definição 7, $c_{\text{eff}}(w) = c_{\text{eff}}(v) \wedge c_w \wedge (\dots)$; logo $\llbracket c_{\text{eff}}(w) \rrbracket = \llbracket c_{\text{eff}}(v) \rrbracket \cap \llbracket c_w \rrbracket \cap \dots \subseteq \llbracket c_{\text{eff}}(v) \rrbracket$. A interseção nunca aumenta o conjunto. $\square$

Teorema 9 (Não-escalada de privilégio). *Em qualquer caminho raiz→folha,*

$$\llbracket c_{\text{eff}}(\text{folha}) \rrbracket \subseteq \llbracket c_{\text{eff}}(\text{pai}) \rrbracket \subseteq \dots \subseteq \llbracket c_{\text{raiz}} \rrbracket = \mathcal{R}.$$

Em particular, nenhum delegado acessa uma linha que o seu delegador não acesse.

Demonstração. Aplicação iterada do Lema 8 ao longo do caminho. $\square$

Corolário 10 (Homomorfismo monótono). *A atribuição $v \mapsto \llbracket c_{\text{eff}}(v) \rrbracket$ é um morfismo monótono da árvore T (ordenada por descendência) para o reticulado $(2^{\mathcal{R}}, \subseteq)$: descer numa aresta corresponde a tomar o meet ($\cap$); a combinação de regras dentro de um nó usa $\cup$ (grants) e $\setminus$ (proibições).*

Teorema 11 (*Fail-closed é o bottom*). *Se não há grant positivo no caminho para (s, a) na folha — isto é, se $G_{\text{folha}}(s, a) = \emptyset$ ou nenhum ancestral concede (s, a) — então $\llbracket c_{\text{eff}} \rrbracket = \emptyset$ e todo acesso é negado.*

Demonstração. $\llbracket c_{\text{eff}} \rrbracket$ é uma interseção que inclui $G_{\text{folha}}(s, a)$ (ou um fator vazio quando um ancestral nega tudo); se algum fator é $\emptyset$, a interseção é $\emptyset$. $\square$

Observação 12. A Definição 7 **unifica** isolamento e permissão: não há “pisso de tenant” separado. O isolamento de tenant é simplesmente o fator c_t introduzido no nó do tenant; *fail-closed* é o *bottom* da álgebra (Teorema 11); o operador é o *top* (sem fator de tenant no caminho).

6 O tenant como corolário (desacoplamento)

Corolário 13 (Isolamento de tenant). *Para todo principal cujo caminho contém o nó do tenant t e nenhum nó com $\top$ acima dele que o ignore, vale $\llbracket c_{\text{eff}} \rrbracket \subseteq \llbracket c_t \rrbracket = \{r : r.\text{tenantId} = t\}$. Logo nenhuma linha de outro tenant é jamais retornada, sem nenhuma regra especial: é consequência direta de c_t estar no caminho.*

Isto justifica formalmente o desacoplamento: removemos a lógica `where:{tenantId}` dos *services* e a substituímos pelo fator c_t propagado. O operador, no *top* ($c = \top$, sem fator de tenant), enxerga todos os tenants — exatamente “eu, no topo, vejo tudo”.

7 Enforcement: pushdown no data layer

A condição efetiva é *empurrada* para dentro da consulta. Numa leitura de `subject s` pelo principal p , o resultado é

$$\text{retornado} = \llbracket c_{\text{eff}}(p, s, \text{read}) \rrbracket \cap \llbracket c_{\text{query}} \rrbracket,$$

onde c_{query} é o `where` pedido pela aplicação. Escritas (`create/update/delete`) são validadas contra $\llbracket c_{\text{eff}}(p, s, a) \rrbracket$; o conjunto de campos ϕ restringe colunas (*whitelist* no `update`, *narrowing* no `select`).

Concretamente, isto é uma *extensão do Prisma Client* (`$allOperations`) que (i) consulta o *resolver* para obter c_{eff} , (ii) injeta-o via **AND** no **where**, e (iii) aplica a política de campos. Há dois modos: **shadow** computa c_{eff} e *registra* (log) onde *negaria*, mas não altera o resultado — para observação segura em produção. **enforce** aplica o *pushdown* e nega de fato.

Observação 14 (Cobertura e o piso nativo). A extensão cobre tudo que passa pelo client estendido. Consultas `$queryRaw` a *escapam*; por isso (a) um teste-*guardrail* falha a CI ao surgir *raw* fora de uma *allowlist* escopada, e (b) numa fase posterior adiciona-se *Row-Level Security* nativa do PostgreSQL (POLICY sobre `current_setting('app.tenant_id')`) como piso duro que captura até o *raw*.

A Plano de implementação

Escopo (backbone). Caso degenerado de dois níveis de propagação: condição de tenant a partir do contexto $\wedge$ grants do perfil. A árvore multi-nível ($\text{org} \rightarrow \text{sub-org}$) entra com o multi-org, mas o desenho já a acomoda.

A1. Resolver de condição. `erp-api/src/access/condition-resolver.ts`: dado (principal, subject, action), computa $c_{\text{eff}} = (\text{escopo herdado}) \wedge (\bigcup \text{grants} \setminus \bigcup \text{proibições})$. Hoje o escopo herdado vem do contexto: $\top$ se operador, senão (`tenantId = user.tenantId`). Reaproveita as regras já carregadas (`buildAbility / access.types.ts`). A extensão não tem lógica de tenant — recebe condição genérica.

A2. Contexto carrega regras + papel (1 query/request). Estender `TenantContext` (`common/tenant-context/tenant-context.ts`) com `accessRules`, `isPlatformOperator`, `inheritedScope` e `rlsMode`. Carregar no `TenantContextInterceptor` com o *client base* (evita recursão). O `PoliciesGuard` passa a reusar as regras do contexto.

A3. Extensão Prisma (genérica). `erp-api/src/prisma/prisma-rls.extension.ts`, adaptada de `easypay-api/src/prisma/prisma.extensions.ts`, porém: sem `tenantId hardcoded`; grants compõem com **OR**; *fail-closed* quando $c_{\text{eff}} = \perp$; mapa *Subject* $\leftrightarrow$ *model* (`Customer` $\rightarrow$ `CustomerSupplier`, `Wallet` $\rightarrow$ `MerchantWallet`, `Role` $\rightarrow$ `AccessRole`, `User` $\rightarrow$ `TenantUser`); *registry* de *models* via DMMF. `PrismaService` vira *Proxy* por-*request* com client estendido em cache.

A4. Provisionamento. `prisma/seed-access.ts`: o perfil “Administrador” do dono = `manage all`; a condição de tenant vem da propagação (contexto), mantendo as regras limpas. *Scaffold* de perfil/condição de operador ($\top$).

A5. Testes (por funcionalidade — exigência central). `prisma-rls.extension.spec.ts` (por subject/model: injeção de c_{eff} , grant permitido/negado, OR de grants, *inverted*, *whitelist* de `fields`, *narrowing* de `select`, *create* cross-scope bloqueado, *fail-closed*, bypass do operador); `condition-resolver.spec.ts` (propagação, monotonicidade, operador= $\top$); `ability.contract.spec.ts` (matriz *can/cannot* de cada perfil-semente — trava regressão em cadeia); e2e 2-tenants no Postgres da CI. Regra de projeto: nenhum perfil/regra novo sem teste.

A6. Gate na CI (antes de cada deploy). `.github/workflows/deploy.yml`: *job* `test` (na *runner*) com `npm ci + npx prisma generate + npm test`; o *job* `deploy` ganha `needs: test`.

A7. Rollout. `RLS_MODE=shadow` primeiro (observação), depois `enforce` (env em `deploy/deploy.sh`). Fase B: *Row-Level Security* nativa do PostgreSQL como piso duro (cuidado com *pooling*), após estabilização.